🚀 Day 10/21: Python Masterclass — File I/O, JSON Parsing & Context Managers

💡 **The Data Refinery Daily Motivation (Day 10)**

"Double-digits day! You've officially crossed into the core engineering phase of our sprint. Keep that momentum going—every concept you master today is a weapon for your next technical interview. Let's Crush Day 10!" ⚡

Author: Prasad S Hiremath

Community: The Data Refinery

🏢 Real-World Scenario: The "Too Many Open Files" Crash at InstaDrop

At **InstaDrop**, a nightly ETL job processes millions of store transaction logs. A junior developer wrote this ingestion script:

Python
```
file = open("instadrop_events.json", "r")
data = file.read()
# Developer forgot file.close()!
```

When running this over thousands of partitioned log files, the Linux server crashed with an `OSError: [Errno 24] Too many open files`. Operating systems limit how many file handles a single process can hold open. If you forget to close files, or if a crash happens midway, the file descriptor remains locked, leaking system resources. In data engineering, we prevent this using **Context Managers (`with` statement)**.

📖 Part 1: Core Concepts Explained

1. **Context Managers (`with` statement):** Think of it like a library automated robotic arm that checks out a book for you and **guarantees** it is safely returned to the shelf the exact second you step out of the room—even if you drop your coffee halfway through!
2. **JSON Parsing (`load` vs `loads`):**
    a. `json.load(file_object)` reads directly from an **open file stream** on disk.

    b. `json.loads(string_data)` parses raw **JSON text strings** (like API responses).
3. **Defensive Log Parsing:** Real-world logs contain corrupted lines due to network dropouts. Always wrap JSON parsing in a `try...except json.JSONDecodeError` block.

🧮 Part 2: File I/O & JSON Quick Reference

Method / Mode	Purpose / Description	Best Practice / Safety Note
`with open(file, 'r') as f:`	Context manager for safe file reading	Automatically closes file handles on exit
`json.load(f)`	Parses JSON directly from a file stream	Use when reading from disk
`json.loads(s)`	Parses JSON from a string variable	Use when ingesting API response payloads
`json.JSONDecodeError`	Exception caught during malformed JSON parsing	Wrap in `try...except` to skip bad rows

💻 Part 3: Coding Challenge & Solution

🛰️ *The Store Log Filter & Parser*

Python

```
import json

def parse_store_logs(log_lines):
    """
    Safely parses JSON log lines and extracts store IDs with ERROR
status.
    Time Complexity: O(N) | Space Complexity: O(K) for error events.
    """
    error_store_ids = []
```

```
    for line in log_lines:
        try:
            event = json.loads(line)
            if event.get("status") == "ERROR":
                error_store_ids.append(event.get("store_id"))
        except (json.JSONDecodeError, TypeError):
            # Gracefully skip corrupted log lines without crashing
            continue

    return error_store_ids

# Execution Sample:
log_lines = [
    '{"store_id": 101, "status": "SUCCESS"}',
    '{"store_id": 102, "status": "ERROR"}',
    'MALFORMED_JSON_LINE_CORRUPT',
    '{"store_id": 103, "status": "ERROR"}'
]
print(parse_store_logs(log_lines))  # Output: [102, 103]
```

❓ Part 4: 5 Scenario-Based Interview MCQs

Q1: Purpose of Context Managers (`with`)

What is the primary advantage of using a context manager when handling files?

- A) It compresses files automatically
- B) It guarantees resource cleanup and prevents file descriptor leaks *(Correct)*
- C) It converts text files into pandas DataFrames
- D) It speeds up CPU processing

Q2: `json.load()` vs `json.loads()`

You have an API response stored in a string variable `response_text`. Which method parses it?

- A) `json.load(response_text)`
- B) `json.loads(response_text)` *(Correct)*

- C) `json.parse(response_text)`
- D) `json.read(response_text)`

Q3: Handling Corrupted Data Files

What exception is raised when attempting to parse a malformed JSON string?

- A) `KeyError`
- B) `TypeError`
- C) `json.JSONDecodeError` *(Correct)*
- D) `FileNotFoundError*`

Q4: File Modes in Python

When opening a file to append new logging records without overwriting existing data, which mode should you use?

- A) `"r"` (Read)
- B) `"w"` (Write)
- C) `"a"` (Append) *(Correct)*
- D) `"x"` (Exclusive)

Q5: Iterating over File Objects

Why is iterating over an open file object line-by-line best practice for large data files?

- A) It loads the entire file into RAM as a single string
- B) It uses lazy iteration, reading one line at a time to maintain a tiny memory footprint *(Correct)*
- C) It deletes lines after reading
- D) It translates lines into SQL queries

Part 5: Bonus Technical Q&A (Interview Prep)

- **Bonus Q1: What is the exact difference between `json.load()` and `json.loads()`?**
    - *Answer:* `json.load()` takes a **file-like object** (an open stream connected to disk or network) and parses the contents.

`json.loads()` (string load) takes a **Python string** containing JSON data. Mixing these up raises a `AttributeError` or `TypeError`.

- **Bonus Q2: How do context managers (`with` statement) work under the hood in Python?**
    - *Answer:* Context managers implement the **Context Manager Protocol**, which requires two special methods on the object: `__enter__()` (executed when entering the `with` block, returning the resource) and `__exit__()` (executed when leaving the block, handling cleanup and exception propagation even if runtime errors occur).

- **Bonus Q3: How do you prevent `UnicodeDecodeError` when ingesting legacy text or log files in Python?**
    - *Answer:* Different systems use different character encodings (e.g., Windows-1252 vs UTF-8). Always specify the encoding explicitly when opening files: `open("file.log", "r", encoding="utf-8", errors="ignore")` or `errors="replace"` to prevent ingestion crashes caused by unexpected special characters.